



CBR-RAG Paper Study and Project Recommendation

A simple Hinglish study of the research paper, six realistic project directions, and a complete recommendation for ResolveAI.

RECOMMENDED PROJECT

ResolveAI

Explainable technical troubleshooting using RAG, CBR and a local LLM

Based on: "CBR-RAG: Case-Based Reasoning for Retrieval Augmented Generation in LLMs for Legal Question Answering"

Executive Summary

Bottom line

The paper offers a useful structured-retrieval idea, but it is not a production-ready blueprint and does not implement the full classical CBR cycle. For a student Major Project, the strongest adaptation is ResolveAI: RAG retrieves authoritative manuals, CBR retrieves past solved technical issues, and a local LLM combines both into cited troubleshooting guidance.

What this document contains

- Paper study: problem, RAG, CBR, LLM role, pipeline, models, dataset, evaluation and limitations.
- Project exploration: six realistic RAG + CBR + LLM ideas across multiple domains.
- Recommendation: exact scope, architecture, stack, team split, timeline and evaluation plan for ResolveAI.

Paper in one sentence

The paper stores previous legal QA examples as structured cases, compares a new question with their questions, evidence and entities using weighted semantic similarity, retrieves the top cases, and gives them to an LLM for final answer generation.

Recommended project in one sentence

ResolveAI uses official technical documentation as RAG knowledge and reviewed solved tickets as a CBR casebase, then uses a local LLM to produce safe, cited and explainable troubleshooting steps.

Decision rationale

Criterion	Why ResolveAI fits
Feasibility	Text-only MVP, local models, small reviewed dataset, no paid API or special hardware.
Technical depth	Dense and sparse retrieval, embeddings, structured case similarity, fusion, prompting, evaluation and full-stack development.
Demo quality	Visible documentation citations, similar-case cards, match reasons, solution steps and feedback retention.
Research value	Clear ablation: LLM-only vs RAG-only vs CBR-only vs combined system.
Viva clarity	RAG is the manual; CBR is technician experience; LLM explains the combined answer.

1. Core Concepts in Simple Hinglish

RAG kya hota hai?

Retrieval-Augmented Generation ko open-book exam samjho. User question poochta hai, retriever relevant document passages laata hai, aur LLM un passages ko use karke answer likhta hai.

User question

>

Relevant passages

>

LLM

>

Grounded answer

RAG analogy

RAG ek technician ka manual hai. Iska strength authoritative evidence hai; limitation yeh hai ki manual har real situation ka exact previous solution nahi batata.

CBR kya hota hai?

Case-Based Reasoning experienced person ki memory jaisa hai: "maine pehle aisa case dekha tha; usmein yeh solution kaam kiya tha."

CBR phase	Simple meaning
Retrieve	New problem ke similar old cases find karo.
Reuse	Old solution ka useful part current situation mein apply karo.
Revise	Result check karke solution correct ya adapt karo.
Retain	Verified new resolution ko future case ke form mein save karo.

Important paper nuance

The paper mainly implements Retrieve and a loose prompt-based form of Reuse. It does not implement explicit adaptation, human revision or verified retention. A student project can improve the paper by completing this four-step loop.

LLM ka role

LLM retrieval engine nahi hai. Retriever evidence and cases select karta hai. LLM selected context ko combine karke readable answer, explanation or checklist banata hai.

Component	Easy analogy	Main responsibility
RAG	Manual/library	Authoritative document evidence retrieve karna
CBR	Experienced technician ki memory	Similar solved situations retrieve karna
LLM	Explainer	Evidence and experience ko coherent answer mein combine karna

2. Paper Problem and Proposed Architecture

Problem being solved

- LLMs can generate fluent but incorrect legal answers or hallucinations.
- Training knowledge may be outdated and difficult to verify.
- Legal answers require provenance and supporting evidence.
- Ordinary RAG usually retrieves unstructured passages using one similarity score and may ignore the structure of previously solved cases.

Structured case representation

Case = <Q, S, E, A>

Q: previous question S: supporting evidence snippet E: entities and identifiers A: previous answer

Dual representations

Representation	Comparison	Purpose
Intra embedding $f()$	New question to old question	Same-field semantic matching
Inter embedding $g()$	New question to support or entities	Information retrieval across different fields

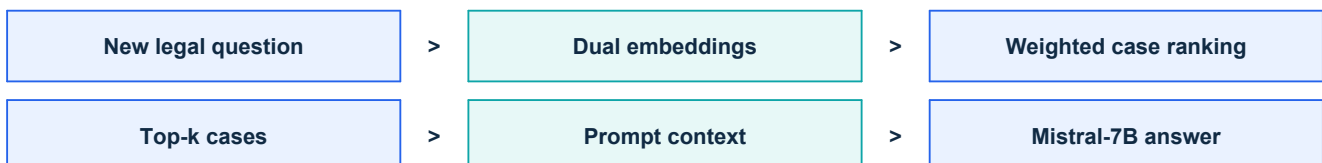
Hybrid retrieval score

Main paper configuration

$0.25 \times \text{question similarity} + 0.40 \times \text{support similarity} + 0.35 \times \text{entity similarity}$

The weights are experimental choices, not universal constants.

End-to-end paper pipeline



Two context strategies were tested:

- Support-only: retrieved evidence passages are given to the LLM.
- Full-case: previous question, support, entities and answer are intended to be included.

3. Complete Workflow and Component Interaction

Offline preparation

Step	What happens
1. Collect	Open Australian Legal QA cases are loaded.
2. Clean	Forty offensive records are removed.
3. Enrich	Legal acts, named entities and unique identifiers are extracted.
4. Embed	Question, support and entity fields are converted to vectors.
5. Store	Structured cases and precomputed representations form the casebase.

Online question answering

Step	What happens
1. Query	A new legal question arrives.
2. Represent	Intra and inter query embeddings are created.
3. Compare	Question-to-question, question-to-support and question-to-entity similarities are computed.
4. Rank	Weighted cosine scores rank all previous cases.
5. Retrieve	Top-k cases are selected.
6. Construct context	Support-only or full-case prompt context is assembled.
7. Generate	Mistral-7B produces the final answer.

Models and techniques

Model / method	Role in the study
GPT-4	Generated the original synthetic ALQA questions and answers from real legal source documents.
GPT-3.5 Turbo	Extracted legal acts, named entities and identifiers.
BERT	General-purpose retrieval representation.
LegalBERT	Legal-domain-pretrained retrieval representation.
AnglEBERT	Contrastive semantic retrieval representation; best result in this study.
Mistral-7B	Generated synthetic test questions and final evaluated answers.
Cosine similarity + kNN	Ranked cases by vector similarity.

4. Dataset, Evaluation and Results

Dataset facts

Item	Value / observation
Source	Open Australian Legal Question Answering (ALQA)
Original records	2,124 question-support-answer examples
Removed	40 offensive records
Final casebase	2,084 structured cases
Legal acts	785 unique acts
No legal-act reference	1,183 cases, reported as 57 percent
Repeated acts	Only 44 acts appeared in more than one case

Test-set construction

- Cases sharing a legal act were paired.
- Mistral-7B generated 35 new questions intended to need both parent cases.
- Manual pruning left only 32 test questions.
- For retrieval evaluation, the two source cases were labelled relevant and every other case irrelevant.

Key results

Evaluation	Best configuration	Result	Correct interpretation
Retrieval	AngIEBERT, weights 0.25 / 0.40 / 0.35, k=3	F1 = 0.300	Relevant parent-case retrieval remained weak.
Generation baseline	No-RAG Mistral-7B	0.8967	Answer-embedding cosine similarity, not correctness.
Best generation	AngIEBERT, k=3, full-case context	0.9141	Absolute gain 0.0174; relative gain about 1.94 percent.

Do not misread 0.9141

This is semantic similarity between generated and reference answer embeddings. It does not mean 91.41 percent legal accuracy. Wrong legal reasoning, unsupported claims or incorrect citations could still receive a high semantic score.

Reported statistical finding

The authors report that the best configuration was significant against No-RAG and one LegalBERT configuration at the 95 percent confidence level. Exact p-values, confidence intervals and effect sizes were not provided.

5. Normal RAG vs the Paper's CBR-RAG

Aspect	Normal RAG	Paper CBR-RAG
Stored knowledge	Document chunks	Structured previous QA cases
Retrieval	Usually one query-to-passage score	Separate question, support and entity scores
Context	Relevant passages	Support-only or previous full cases
Strength	Authoritative facts	Fact evidence plus analogous experience
Risk	Passage may not directly solve the situation	Old answer may be copied or adapted incorrectly
Learning	Usually static	Paper is also static; no real verified retention

Fair conclusion
The paper shows that retrieved structured cases can help compared with no retrieval. It does not conclusively show that CBR-RAG beats a strong standard RAG pipeline with BM25, dense retrieval and reranking.

Major limitations and threats to validity

Limitation	Why it matters
Incomplete CBR	No explicit adaptation, revision, validation or retention.
Tiny synthetic test set	Only 32 generated questions; weak generalization evidence.
Circular model use	Mistral created test QA and also generated evaluated answers.
Narrow metric	Embedding cosine does not measure legal correctness or citation faithfulness.
No legal expert review	Real-world usefulness and safety remain unverified.
No strong RAG baseline	The specific benefit of the CBR formulation is not isolated.
Low retrieval F1	Best parent-case retrieval F1 was only 0.300.
Selection and tuning risk	Shared-act questions and hyperparameter tuning used the same small set.
Reproducibility gaps	Exact checkpoints, pooling, generation settings, seeds and hardware are missing.
Domain limitation	Australian legal data does not transfer directly to Indian law.

6. Realistic Major Project Ideas

Common architecture
RAG knowledge base: authoritative documents. CBR casebase: verified past problem-solution records. LLM: explanation and adaptation. Feedback: only reviewed outcomes are retained.

Idea 1 - ResolveAI: Technical Troubleshooting Copilot

Problem and users: Students, developers and lab staff repeatedly search manuals and old forum threads for Python, Git, Linux or Docker errors.

Technology roles: RAG retrieves official manuals; CBR finds similar solved tickets using error, tool, OS and symptoms; the LLM creates cited troubleshooting steps.

Issue / log	>	Docs + solved cases	>	Cited resolution	>	Verified retention
Data	Complexity	Strong demo	Limit / extension			
300-600 reviewed accepted Q&A; records and official docs	Medium; 8-12 weeks	Show citations, similar cases, match reasons and safe steps	Keep domain narrow; later add logs, screenshots and helpdesk integration			

Idea 2 - CodeCase: Repository-Aware Debugging Assistant

Problem and users: Developers search source code, docs, issues, pull requests and fixes separately.

Technology roles: RAG retrieves code and documentation; CBR retrieves similar resolved issues and linked commits; the LLM explains probable root cause and proposes a testable patch.

Bug / stack trace	>	Code + issue retrieval	>	Root cause / patch	>	Developer validation
Data	Complexity	Strong demo	Limit / extension			
One mature open-source repository, docs, closed issues and linked fixes	Medium-high; 10-14 weeks	Show relevant file, similar issue and associated commit	Issue-to-fix links are noisy; later add sandboxed tests and PR drafting			

6. Project Ideas - Continued

Idea 3 - CampusCompass: Academic Policy Advisor

Problem and users: Students misunderstand attendance, medical leave, backlog, scholarship and examination rules.

Technology roles: RAG retrieves official policy clauses; CBR retrieves similar anonymized and verified student situations; the LLM explains the rule and produces a document checklist.

Student situation	>	Policy + similar cases	>	Cited explanation	>	Staff verification
Data		Complexity		Strong demo		Limit / extension
College handbooks, notices, forms and 50-100 verified scenarios		Low-medium; 6-10 weeks		Attendance scenario with exact clause and action checklist		Policies change and the system cannot make official decisions

Idea 4 - SecureCase: Defensive Cybersecurity Assistant

Problem and users: Learners and small security teams struggle to connect logs or suspicious emails with standards and previous incidents.

Technology roles: RAG retrieves NIST and MITRE guidance; CBR finds similar incidents using indicators, assets and outcomes; the LLM generates a defensive triage checklist and report.

Sanitized log / email

>

Guidance + incidents

>

Defensive triage

>

Analyst review

Data	Complexity	Strong demo	Limit / extension
MITRE ATT&CK, NIST guidance and controlled lab incidents	High; 12-16 weeks	Map indicators to ATT&CK; and show similar incidents	Expert validation required; keep strictly defensive

6. Project Ideas - Continued

Idea 5 - FinResolve: Consumer Complaint Assistant

Problem and users: Consumers struggle to organize complaints, find the correct procedure and understand similar outcomes.

Technology roles: RAG retrieves official regulator procedures; CBR retrieves similar product or service complaints; the LLM creates a cited checklist and editable complaint draft.

Data	Complexity	Strong demo	Limit / extension
CFPB complaint records and official complaint guidance	Medium; 8-12 weeks	Classify a complaint and show related cases and a draft	US-specific and sensitive; no legal or financial advice

Idea 6 - MaintainIQ: Equipment Maintenance Assistant

Problem and users: Lab technicians search long manuals while past repair knowledge remains scattered.

Technology roles: RAG retrieves manuals and SOPs; CBR finds faults with similar model, error code and repair outcome; the LLM generates a safe diagnostic sequence and report.

Data	Complexity	Strong demo	Limit / extension
Public manuals and verified cases for 2-3 equipment families	Medium; 8-12 weeks	Exact manual section, similar repairs and ordered checks	Case collection is difficult; later add images and sensors

Comparison

Project	Feasibility	Depth	Demo	Data	Main risk
ResolveAI	5/5	4.5/5	5/5	4.5/5	Scope becoming too broad
CodeCase	4/5	5/5	4.5/5	4/5	Issue-to-fix linking
CampusCompass	4.5/5	3.5/5	4.5/5	3/5	Verified cases
SecureCase	3/5	5/5	5/5	4/5	Complexity and validation
FinResolve	4/5	4/5	4/5	5/5	Sensitive / US-specific
MaintainIQ	4/5	4/5	4.5/5	2.5/5	Case collection

7. Recommended Project: ResolveAI

Formal title
ResolveAI: An Explainable Technical Troubleshooting Assistant Using Hybrid Retrieval-Augmented Generation, Case-Based Reasoning and a Local Large Language Model

Core research question

Research focus
Does combining authoritative documentation with structured past solved cases improve troubleshooting accuracy, citation quality and resolution usefulness compared with an LLM-only or RAG-only system?

Why it wins

- Technical support tickets are naturally genuine cases, so CBR is not forced.
- RAG and CBR have visibly different jobs: manuals versus past successful experience.
- Free documentation and public accepted-answer data are available.
- The demo is understandable even to a non-technical evaluator.
- It covers NLP, embeddings, dense and sparse retrieval, reranking, prompting, evaluation, backend and frontend.
- A feedback gate extends the paper into the complete Retrieve-Reuse-Revise-Retain cycle.

Frozen MVP scope

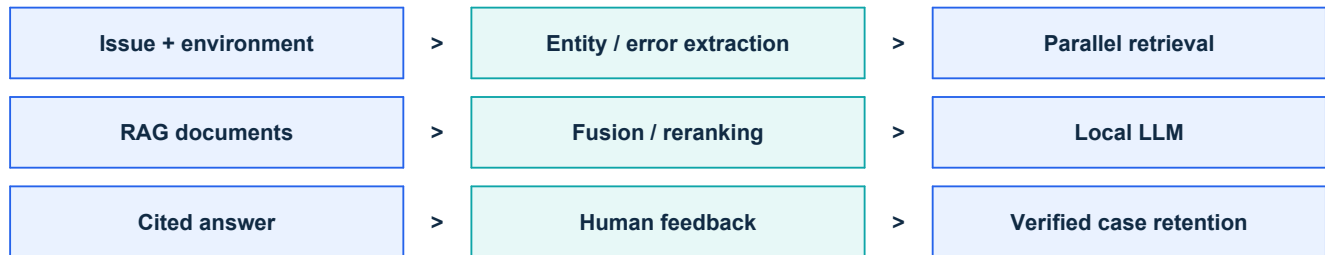
Include first	Add only if stable	Do not promise initially
Python packaging, pip and virtual environments; Git errors; text input; citations; similar-case cards; feedback	Docker and Linux environment issues; reranker; multilingual answers	Autonomous command execution; every language or OS; live crawling; voice; screenshots; multi-agent repair

Final prototype screens

Screen	What the evaluator sees
Issue submission	Problem description, operating system, tool/version and optional error log.
Resolution	Probable diagnosis, clarification, ordered steps and confidence warning.
Evidence	Official document citations and retrieved passages.
Similar cases	Top cases, match score and explanation of why each matched.
Feedback	Solved/not-solved status, corrected resolution and verification flow.
Admin/evaluation	Document upload, case review, weight settings and comparison metrics.

8. ResolveAI Architecture and Technology Stack

Online workflow



Structured case schema

Case fields

problem, tool, operating system, version, error code, symptoms, attempted steps, supporting evidence, verified solution, outcome, source URL and verification status.

Transparent CBR score

Example starting formula

$0.40 \times \text{problem semantic similarity} + 0.25 \times \text{evidence similarity} + 0.20 \times \text{entity/error overlap} + 0.15 \times \text{environment/tool match}$

Weights must be tuned and reported through experiments.

Recommended stack

Layer	Choice	Reason
Frontend	React + TypeScript + Vite	Polished demo and strong resume value.
Backend	Python + FastAPI	Natural ML ecosystem and automatic API documentation.
Application data	SQLite	Users, cases, feedback and audit history without cloud cost.
Lexical retrieval	SQLite FTS5 / BM25	Keyword and error-code matching.
Vector store	Chroma local	Persistent local embeddings and metadata filtering.
Embeddings	BGE-small; MiniLM fallback	Lightweight semantic retrieval.
Local LLM	Qwen3 4B via Ollama	No paid generation API; 1.7B fallback for low RAM.
Reranking	MiniLM CrossEncoder, optional	Improves top-candidate ordering after MVP works.

Viva implementation principle

Implement the core retrieval, scoring and fusion in transparent Python rather than hiding the complete system behind a large framework. Every score and retrieved item should be inspectable.

9. Data, Team Division and Timeline

Practical data plan

- RAG corpus: official Python packaging, Git and selected tool documentation.
- CBR casebase: 300-600 reviewed accepted-answer records with attribution plus team-created verified test cases.
- Evaluation: 50-100 held-out issues with known relevant documents, cases and expected resolution points.
- Quality rule: automatically generated chat responses must never be retained without human approval.

Team division

Member	Primary responsibility	Main deliverables
Member 1	Data and RAG	Collection, cleaning, chunking, embeddings, BM25, citations and retrieval evaluation.
Member 2	CBR and backend	Case schema, similarity engine, fusion, revision/retention and FastAPI.
Member 3	LLM and frontend	Ollama integration, prompting, React UI, feedback and dashboard.
All members	Integration and academic work	Testing, experiment set, documentation, report, presentation and viva.

Realistic semester plan

Period	Target
Week 1	Freeze scope, define case schema and collect starter data.
Week 2	Build basic document ingestion and RAG retrieval.
Week 3	Build CBR retrieval and transparent weighted scoring.
Week 4 - mid-sem	Basic UI showing answer evidence and similar cases.
Weeks 5-6	Local LLM, grounded output format and citation handling.
Weeks 7-8	Feedback, revision, verified retention and admin flow.
Weeks 9-10	Polished UI, reranking if needed and evaluation dashboard.
Weeks 11-12	Ablation, testing, optimization, report and viva preparation.

10. Evaluation, Feasibility and Major-Project Fit

Required experiment

System version	Purpose
LLM only	Measure what the local model can answer without evidence.
RAG only	Measure the value of authoritative document retrieval.
CBR only	Measure the value of previous solved experience.
RAG + CBR	Test the central project hypothesis.
RAG + CBR + reranker	Optional advanced configuration after the core system works.

Metrics

Area	Suggested metrics
Document retrieval	Recall@3/5, MRR, nDCG
Case retrieval	Relevant-case Top-1 and Top-3 accuracy
Answer quality	Correctness rubric, faithfulness, relevance and citation precision
CBR lifecycle	Reuse acceptance rate, revision rate and verified retention count
System quality	Latency, peak memory, abstention rate and failed-query analysis

Why almost zero budget

- Open documentation and public or team-curated cases.
- Local embeddings, local vector storage and SQLite.
- Local quantized LLM through Ollama; no paid generation API.
- No GPU is compulsory; CPU generation will be slower but acceptable for a controlled demo.
- Models can be downloaded before the viva so the demonstration works offline.

Complex engineering mapping

Criterion	Project mapping
WP1 / WP3	Engineering knowledge and experimental design for embeddings, retrieval, similarity and fusion.
WP2	Accuracy, latency, privacy, content licensing and unsafe-command trade-offs.
WP6 / WP7	Users, support staff and administrators across a multi-component system.
EA1-EA3	Diverse data/software resources and a research-based hybrid solution.
EA4 / EA5	Wrong advice has consequences; confidence, verification and principles-based evaluation are needed.

11. Final Recommendation

Choose ResolveAI

It offers the best balance of technical depth, semester feasibility, live-demo quality, research potential, resume value and viva explainability. The first working version should remain limited to Python/package-management and Git problems.

What makes it more than a chatbot

- Two different knowledge sources: authoritative documents and verified experience.
- Transparent, multi-field case similarity rather than one generic vector search.
- Visible evidence, citations, match explanations and confidence handling.
- A complete human-controlled CBR cycle with revision and retention.
- A proper experimental comparison rather than only a polished interface.

What must remain realistic

- Do not claim support for every technology or operating system.
- Do not execute generated commands automatically.
- Do not retain unverified LLM outputs as cases.
- Do not claim that semantic similarity equals correctness.
- Do not make a paid API or powerful GPU a project requirement.

Useful sources

Resource	Link
Base paper	https://arxiv.org/abs/2404.04302
Authors' code	https://github.com/rgu-iit-bt/cbr-for-legal-rag
ALQA dataset	https://huggingface.co/datasets/isaacus/open-australian-legal-qa
IBM TechQA	https://research.ibm.com/publications/the-techqa-dataset
Stack Exchange API	https://api.stackexchange.com/docs/types/question
Stack Overflow licensing	https://stackoverflow.com/help/licensing
Ollama local API	https://docs.ollama.com/api/introduction
BGE-small model card	https://huggingface.co/BAAI/bge-small-en-v1.5

Next decision

After project approval, the next deliverables should be a college-ready synopsis, frozen MVP specification, data-source plan, detailed system design and implementation backlog before coding begins.